

# 浙江大学



## 本科实验报告

|      |           |
|------|-----------|
| 姓名   |           |
| 学院   | 控制科学与工程学院 |
| 系    | 自动化系      |
| 专业   | 自动化（控制）   |
| 学号   |           |
| 指导教师 | 林峰、韩涛     |

2026 年 5 月 28 日

# 浙江大学实验报告

课程名称: 控制与决策系统仿真      实验类型: 上机实验

实验项目名称: 第五章实验

学生姓名: \_\_\_\_\_ 专业: 自动化(控制) 学号: \_\_\_\_\_

同组学生姓名:

指导教师: 林峰、韩涛                      实验地点: 东三-411

实验日期: 2026 年 5 月 28 日

## 1 实验目的和要求

1. 熟悉分类、聚类 and 关联等数据挖掘操作原理。
2. 了解 MATLAB 实现分类、聚类 and 关联操作的方法。
3. 掌握数据挖掘中数据预处理的一般流程，并能用程序举例说明。
4. 编程实现 CART 决策树数据分类，并能对 Iris 数据集分类结果进行分析。

## 2 实验内容和原理

本次实验完成两道题：说明数据挖掘中数据处理的一般过程并编程举例；以 Iris 数据集为对象实现 CART 决策树分类。

### 3 主要仪器设备

Windows 10、MATLAB 2022b、Office 2010 软件。

## 4 操作方法与实验步骤

在 MATLAB 中分别编写数据预处理示例脚本和 CART 分类脚本，运行后记录命令行输出并保存图像，详见第五节。

## 5 实验数据记录和处理

### 5.1 数据挖掘中的数据处理过程及程序举例

**问题分析：**数据挖掘的数据处理位于建模之前，其作用是把原始数据整理为结构清楚、数值范围合适的数据。一般流程包括数据获取、数据清洗、数据集成、数据变换、数据规约、数据划分和结果检查等环节，其中清洗与变换会直接影响后续分类或聚类结果。

具体过程可概括为：

1. 数据获取：从实验数据文件、数据库或 MATLAB 内置数据集中读取样本、特征和标签。
2. 数据理解：检查样本数量、特征维数、变量类型、类别分布和异常范围。
3. 数据清洗：处理缺失值、重复样本、异常值和明显错误记录。
4. 数据集成：把来自不同来源或不同表格的数据按样本编号或时间顺序合并。
5. 数据变换：进行标准化、归一化、类别编码、特征构造等操作。
6. 数据规约：根据任务需要筛选特征、降维或抽样，以降低计算复杂度。
7. 数据划分：将数据分为训练集、验证集和测试集，避免用测试数据参与训练。
8. 建模准备与检查：输出处理后的数据规模、类别比例和可视化图，确认数据格式和数量无误。

程序代码：

```
1 %% 题目1: 数据挖掘数据处理流程示例
2 % 示例使用 Iris 数据集，演示缺失值处理、标准化、标签编码、
3 % 训练测试划分和简单统计汇总。
4
5 clear; clc; close all;
6
7 scriptDir = fileparts(mfilename('fullpath'));
8 addpath(scriptDir);
9 [X, Y] = load_iris_local(scriptDir);
10
11 % 人为构造少量缺失值，用于演示缺失值填补流程
12 rng(1);
13 missingIndex = randperm(numel(X), 8);
14 X(missingIndex) = NaN;
15
16 % 1. 缺失值处理：用每列均值填补
17 X_fill = X;
18 colMean = mean(X, 'omitnan');
19 for j = 1:size(X, 2)
20     idx = isnan(X_fill(:, j));
21     X_fill(idx, j) = colMean(j);
22 end
23
```

```

24 % 2. 标准化: 消除不同量纲对后续模型的影响
25 mu = mean(X_fill, 1);
26 sigma = std(X_fill, 0, 1);
27 X_norm = (X_fill - mu) ./ sigma;
28
29 % 3. 标签编码: 将类别型标签转换为分类变量和整数编号
30 [classNames, ~, Y_id] = unique(Y);
31
32 % 4. 数据划分: 70% 训练集, 30% 测试集
33 rng(2);
34 trainMask = false(size(Y_id));
35 for c = 1:numel(classNames)
36     idx = find(Y_id == c);
37     idx = idx(randperm(numel(idx)));
38     nTrain = round(0.70 * numel(idx));
39     trainMask(idx(1:nTrain)) = true;
40 end
41 testMask = ~trainMask;
42 X_train = X_norm(trainMask, :);
43 Y_train = Y(trainMask);
44 X_test = X_norm(testMask, :);
45 Y_test = Y(testMask);
46
47 % 5. 统计汇总: 查看处理后数据规模和各类别数量
48 fprintf('原始样本数: %d\n', size(X, 1));
49 fprintf('特征维数: %d\n', size(X, 2));
50 fprintf('缺失值个数: %d\n', sum(isnan(X), 'all'));
51 fprintf('训练集样本数: %d\n', size(X_train, 1));
52 fprintf('测试集样本数: %d\n', size(X_test, 1));
53
54 disp('类别名称: ');
55 disp(classNames);
56 disp('各类别样本数: ');
57 for c = 1:numel(classNames)
58     fprintf('%s: %d\n', classNames{c}, sum(Y_id == c));
59 end
60
61 % 6. 可视化: 绘制标准化后的前两个特征散点图
62 figure;
63 hold on;
64 markers = {'o', 's', '^'};
65 colors = lines(numel(classNames));
66 for c = 1:numel(classNames)
67     idx = Y_id == c;
68     plot(X_norm(idx, 1), X_norm(idx, 2), markers{c}, ...
69         'Color', colors(c, :), 'MarkerFaceColor', colors(c, :), ...
70         'DisplayName', classNames{c});
71 end
72 hold off;

```

```
73 xlabel('标准化萼片长度');  
74 ylabel('标准化萼片宽度');  
75 title('数据预处理后的 Iris 样本分布');  
76 legend('Location', 'best');  
77 grid on;
```

运行结果：

原始样本数：150  
特征维数：4  
缺失值个数：8  
训练集样本数：105  
测试集样本数：45  
类别名称：  
    {'setosa' }  
    {'versicolor'}  
    {'virginica' }  
  
各类别样本数：  
setosa: 50  
versicolor: 50  
virginica: 50

预处理后的样本分布如图所示。

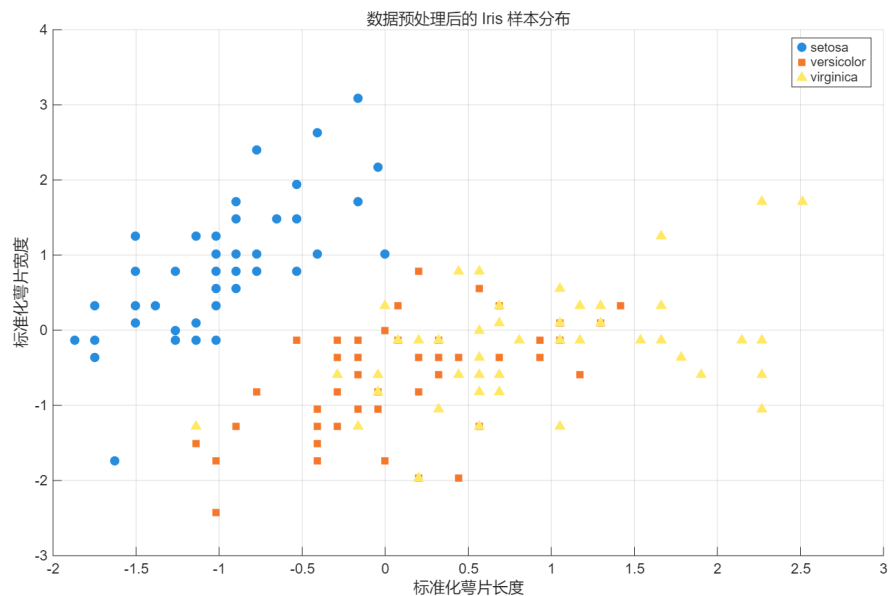


图 1: 数据预处理后的 Iris 样本分布

**结果分析：**Iris 数据集共有 150 个样本、4 个特征，三类样本数量均为 50，类别分布均衡。实验中设置的 8 个缺失值均被检测到，之后按 70% 和 30% 分层划分为 105 个训练样本和 45 个测试样本，样本总数与原始数据一致。由散点图可见，setosa 与另外两类在前两个标准化特征上的分布差异较明显，而 versicolor 与 virginica 有一定重叠。

## 5.2 CART 决策树实现 Iris 数据分类

**问题分析：**CART 分类树通过递归划分特征空间建立树结构，每个内部节点选择一个特征及阈值，使划分后的类别纯度提高。本题采用 Gini 指数作为划分准则，并根据训练集递归生成分类树。分类树常用 Gini 指数衡量节点不纯度：

$$Gini(D) = 1 - \sum_{k=1}^K p_k^2,$$

其中  $p_k$  表示数据集  $D$  中第  $k$  类样本所占比例。

**程序代码：**

```

1 %% 题目2: CART 决策树实现 Iris 数据分类
2 % 使用 Iris 数据集并根据 Gini 指数建立 CART 分类树。
3
4 clear; clc; close all;
5
6 scriptDir = fileparts(mfilename('fullpath'));
7 addpath(scriptDir);
8 [X, species] = load_iris_local(scriptDir);
9 [classNames, ~, Y] = unique(species);
10 featureNames = {'SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth'};
11
12 % 按类别分层划分训练集和测试集
13 rng(2);
14 trainMask = false(size(Y));
15 for c = 1:numel(classNames)
16     idx = find(Y == c);
17     idx = idx(randperm(numel(idx))));
18     nTrain = round(0.70 * numel(idx));
19     trainMask(idx(1:nTrain)) = true;
20 end
21 testMask = ~trainMask;
22 X_train = X(trainMask, :);
23 Y_train = Y(trainMask);
24 X_test = X(testMask, :);
25 Y_test = Y(testMask);
26
27 % 训练 CART 分类树。
28 maxDepth = 5;
29 minLeafSize = 2;
30 tree = build_cart_tree(X_train, Y_train, maxDepth, minLeafSize, numel(classNames));
31
32 % 在测试集上预测并计算准确率
33 Y_pred = predict_cart_tree(tree, X_test);
34 accuracy = mean(Y_pred == Y_test);
35 confMat = zeros(numel(classNames));
36 for i = 1:numel(Y_test)
37     confMat(Y_test(i), Y_pred(i)) = confMat(Y_test(i), Y_pred(i)) + 1;

```

```

38 end
39
40 fprintf('测试集样本数: %d\n', numel(Y_test));
41 fprintf('分类正确样本数: %d\n', sum(Y_pred == Y_test));
42 fprintf('测试集准确率: %.4f\n', accuracy);
43 disp('混淆矩阵: ');
44 disp(confMat);
45
46 % 输出每个样本的预测结果, 便于检查分类效果
47 disp('前 15 个测试样本预测结果: ');
48 for i = 1:min(15, numel(Y_test))
49     fprintf('%2d 真实类别: %-10s 预测类别: %-10s 正确: %d\n', ...
50         i, classNames{Y_test(i)}, classNames{Y_pred(i)}, Y_pred(i) == Y_test(i));
51 end
52
53 % 打印决策树结构
54 fprintf('\nCART 决策树结构: \n');
55 print_cart_tree(tree, featureNames, classNames, 0);
56
57 figure;
58 imagesc(confMat);
59 colormap(flipud(gray));
60 colorbar;
61 axis equal tight;
62 set(gca, 'XTick', 1:numel(classNames), 'XTickLabel', classNames, ...
63     'YTick', 1:numel(classNames), 'YTickLabel', classNames);
64 xlabel('预测类别');
65 ylabel('真实类别');
66 title('CART 决策树 Iris 分类混淆矩阵');
67 for i = 1:numel(classNames)
68     for j = 1:numel(classNames)
69         text(j, i, num2str(confMat(i, j)), ...
70             'HorizontalAlignment', 'center', 'Color', 'red', 'FontWeight', 'bold');
71     end
72 end
73
74 % 用花瓣长度和花瓣宽度绘制测试样本预测结果
75 figure;
76 hold on;
77 markers = {'o', 's', '^'};
78 colors = lines(numel(classNames));
79 for c = 1:numel(classNames)
80     idx = Y_pred == c;
81     plot(X_test(idx, 3), X_test(idx, 4), markers{c}, ...
82         'Color', colors(c, :), 'MarkerFaceColor', colors(c, :), ...
83         'DisplayName', classNames{c});
84 end
85 hold off;
86 xlabel('PetalLength');

```

```

87 ylabel('PetalWidth');
88 title('CART 决策树测试集预测分类结果');
89 legend('Location', 'best');
90 grid on;
91
92 function node = build_cart_tree(X, Y, maxDepth, minLeafSize, nClasses)
93     node.class = mode(Y);
94     node.isLeaf = true;
95     node.feature = [];
96     node.threshold = [];
97     node.left = [];
98     node.right = [];
99
100     if maxDepth == 0 || numel(Y) <= minLeafSize || numel(unique(Y)) == 1
101         return;
102     end
103
104     [bestFeature, bestThreshold, bestGain] = best_split(X, Y, minLeafSize, nClasses);
105     if isempty(bestFeature) || bestGain <= 0
106         return;
107     end
108
109     leftIdx = X(:, bestFeature) <= bestThreshold;
110     rightIdx = ~leftIdx;
111     node.isLeaf = false;
112     node.feature = bestFeature;
113     node.threshold = bestThreshold;
114     node.left = build_cart_tree(X(leftIdx, :), Y(leftIdx), maxDepth - 1, minLeafSize,
115         nClasses);
116     node.right = build_cart_tree(X(rightIdx, :), Y(rightIdx), maxDepth - 1, minLeafSize,
117         nClasses);
118 end
119
120 function [bestFeature, bestThreshold, bestGain] = best_split(X, Y, minLeafSize, nClasses)
121     bestFeature = [];
122     bestThreshold = [];
123     bestGain = 0;
124     parentGini = gini_index(Y, nClasses);
125
126     for f = 1:size(X, 2)
127         values = unique(X(:, f));
128         if numel(values) < 2
129             continue;
130         end
131         thresholds = (values(1:end-1) + values(2:end)) / 2;
132         for t = thresholds'
133             leftIdx = X(:, f) <= t;
134             rightIdx = ~leftIdx;
135             if sum(leftIdx) < minLeafSize || sum(rightIdx) < minLeafSize

```



```

134         continue;
135     end
136     leftGini = gini_index(Y(leftIdx), nClasses);
137     rightGini = gini_index(Y(rightIdx), nClasses);
138     childGini = sum(leftIdx) / numel(Y) * leftGini + sum(rightIdx) / numel(Y) *
rightGini;
139     gain = parentGini - childGini;
140     if gain > bestGain
141         bestGain = gain;
142         bestFeature = f;
143         bestThreshold = t;
144     end
145 end
146 end
147 end
148
149 function g = gini_index(Y, nClasses)
150     g = 1;
151     for c = 1:nClasses
152         p = sum(Y == c) / numel(Y);
153         g = g - p^2;
154     end
155 end
156
157 function Y_pred = predict_cart_tree(tree, X)
158     Y_pred = zeros(size(X, 1), 1);
159     for i = 1:size(X, 1)
160         Y_pred(i) = predict_one(tree, X(i, :));
161     end
162 end
163
164 function y = predict_one(node, x)
165     if node.isLeaf
166         y = node.class;
167     elseif x(node.feature) <= node.threshold
168         y = predict_one(node.left, x);
169     else
170         y = predict_one(node.right, x);
171     end
172 end
173
174 function print_cart_tree(node, featureNames, classNames, depth)
175     indent = repmat(' ', 1, depth);
176     if node.isLeaf
177         fprintf('%s预测类别: %s\n', indent, classNames{node.class});
178     else
179         fprintf('%s如果 %s <= %.4f:\n', indent, featureNames{node.feature}, node.threshold);
180         print_cart_tree(node.left, featureNames, classNames, depth + 1);
181         fprintf('%s否则:\n', indent);

```

```

182     print_cart_tree(node.right, featureNames, classNames, depth + 1);
183     end
184 end

```

### 运行结果：

测试集样本数：45

分类正确样本数：45

测试集准确率：1.0000

混淆矩阵：

|    |    |    |
|----|----|----|
| 15 | 0  | 0  |
| 0  | 15 | 0  |
| 0  | 0  | 15 |

前 15 个测试样本预测结果：

|    |             |             |      |
|----|-------------|-------------|------|
| 1  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 2  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 3  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 4  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 5  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 6  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 7  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 8  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 9  | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 10 | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 11 | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 12 | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 13 | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 14 | 真实类别：setosa | 预测类别：setosa | 正确：1 |
| 15 | 真实类别：setosa | 预测类别：setosa | 正确：1 |

CART 决策树结构：

如果 PetalLength <= 2.4500:

    预测类别：setosa

否则：

    如果 PetalWidth <= 1.7500:

        如果 PetalLength <= 4.9500:

            如果 SepalLength <= 5.0000:

                预测类别：versicolor

            否则：

                预测类别：versicolor

        否则：

            如果 PetalWidth <= 1.5500:

                预测类别：virginica

            否则：

                预测类别：versicolor

    否则：

        如果 PetalLength <= 4.8500:

            预测类别：virginica

否则:

预测类别: **virginica**

混淆矩阵图和测试集预测散点图如下。

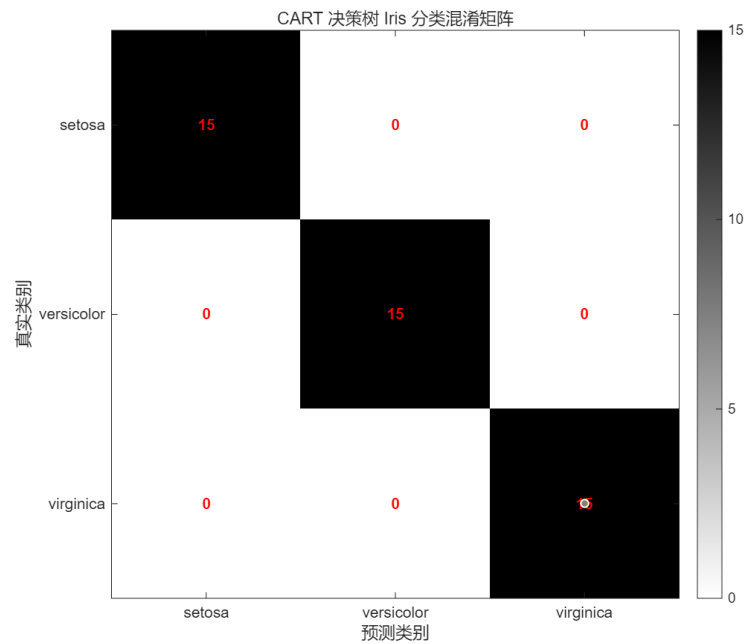


图 2: CART 决策树 Iris 分类混淆矩阵

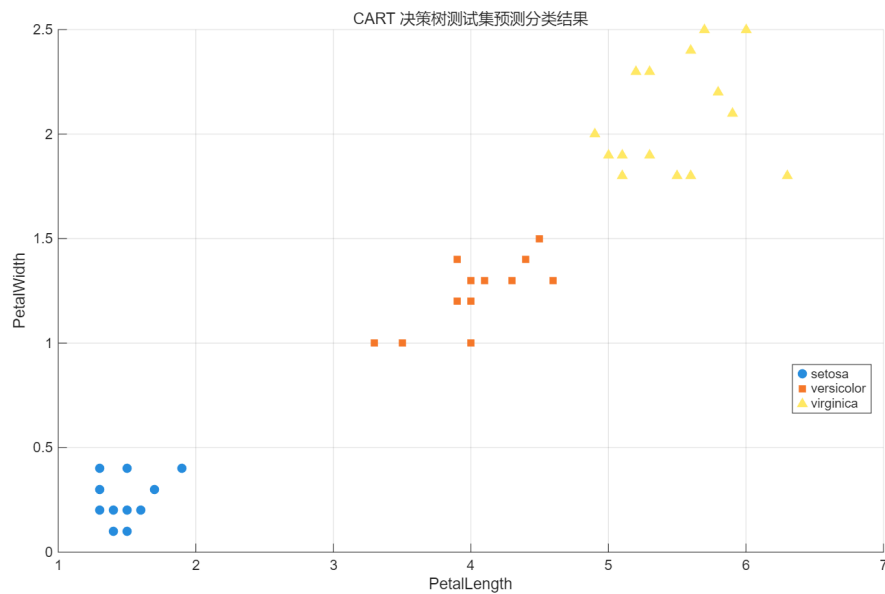


图 3: CART 决策树测试集预测分类结果

**结果分析:** 测试集中共有 45 个样本, 45 个样本均分类正确, 测试集准确率为 1.0000。混淆矩阵的非零元素全部位于主对角线上, 表明 setosa、versicolor 和 virginica 三类在本次测试集中均被正确识别。决策树先以 PetalLength 是否小于等于 2.4500 区分 setosa, 再根据 PetalWidth 和 PetalLength 对 versicolor 与 virginica 继续划分, 这与 Iris 数据集中花瓣特征区分度较高的特点一致。

## 6 实验结果与分析

1. 数据挖掘的数据处理包括读取、清洗、变换、划分和检查等步骤，各步骤共同决定后续建模质量。
2. 缺失值填补和标准化能够提高输入数据的一致性，避免量纲差异干扰分类边界。
3. CART 决策树通过 Gini 指数选择划分特征，树结构直观，便于解释每次分类依据。
4. Iris 数据集中花瓣长度和花瓣宽度对分类贡献较大，本次分类树也主要围绕这两个特征进行划分。

## 7 讨论、心得

本次实验先完成 Iris 数据的缺失值处理、标准化和训练测试划分，再用 CART 决策树完成分类。实验结果表明，setosa 与其他两类较容易区分，versicolor 与 virginica 的边界相对接近，因此花瓣长度和花瓣宽度成为决策树中的关键划分特征。CART 决策树的分类过程直观，便于结合树结构和混淆矩阵分析分类结果。